Développement Web:
Javascript & PHP

PARTIE 3: PHP

LA PROGRAMMATION ORIENTÉ OBJET

La POO

- Un objet est toute donnée manipulée
  - la voiture de mon voisin , mon compte bancaire
- Chaque objet a des attributs qui lui sont propres
  - Mon compte bancaire :
    - Le numéro du compte
    - Le solde actuel
    - Liste des différentes opérations
- Les objets peuvent avoir des méthodes
  - Ce sont des actions que l'on peut appliquer à un objet
  - Mon compte bancaire :
    - Créditer,
    - Débiter, etc

Classes et instances

- Une classe est un modèle de donnée
  - On peut la voir comme une famille d'objets
  - Tous les objets de la classe sont similaires
    - Partagent les mêmes attributs et les mêmes méthodes
- **Exemple**
- Classe rassemblant toutes les voitures
  - Attributs
    - Toutes les voitures (les objets) ont des plaques d'immatriculation, un moteur, un nombre de portes…
  - Méthodes
    - Toutes les voitures (les objets) ont des méthodes pour démarrer, freiner, accélérer…

Classes et instances

- Une instance est une représentation particulière d'une classe
- **Exemple**
  - Voiture est une classe
  - La voiture que vous venez d'acheter est une instance
    - Elle est bleue, tout options
- Une autre instance est :
    - la voiture rouge garée en bas de chez vous

Déclaration

- Déclarer une classe

```
class person {
  //contenu de la classe
}
```

- Le contenu de la classe est structuré en deux parties
  - La première partie permet de déclarer les attributs
  - La deuxième partie du contenu d'une classe permet la déclaration des méthodes

Déclaration

- **Les attributs**
- Les attributs sont déclarés en utilisant la syntaxe des variables et un des mots-clés suivant : public, private ou protected

```
class person {
    private $name;
}
```

- Il est possible d'utiliser une valeur par défaut

```
class person {
    private $name ='user';
}
```

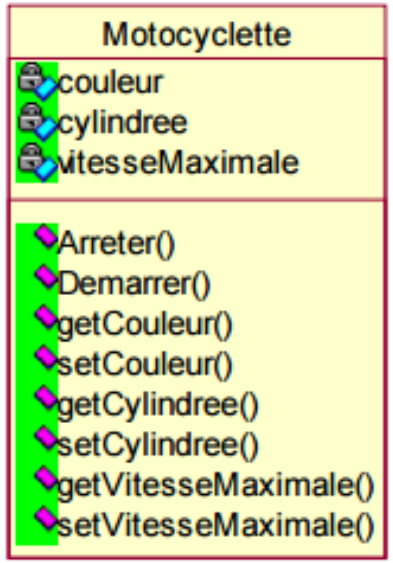
Déclaration

- **Les méthodes**
- Ces méthodes se déclarent exactement comme des fonctions :

```
class person {
private $name;
function set_name($new_name) {
    $this->name = $new_name; }

function get_name() {
    return $this->name;  }
}
```

Exemple




```php
class Motocyclette
{
    //attributs
    private $couleur;
    private $cylindree;
    private $vitesseMaximale;

    //méthodes
    public function getCouleur()
    {
        return $this->couleur;
    }

    public function setCouleur($couleur)
    {
        $this->couleur = $couleur;
    }

    //etc …
}
```

Instanciation

- **Créer un objet**
- créer une classe n'est pas suffisant pour pouvoir l'utiliser
- Pour l'utiliser proprement et avoir accès à ses paramètres, il faut en créer des instances : objets

```
$var_objet = new nom_classe() ;
```

- le code de définition de la classe doit être dans le même script ou inclus au début du script à l'aide des fonctions **`require()`** ou **`include()`**

Instanciation

- **Accès aux propriétés d'un objet**
- Pour accéder, aussi bien en lecture qu'en écriture, à la propriété d'un objet, on utilise : **->**

```
$nom_objet->prop;
```

- si la propriété **prop** de l'objet est un tableau

```
$nom_objet-> prop[n];
```

- **Pour appeler une méthode de l'objet**
- appliquez la même notation :

```
$nom_objet->nom_fonction();
```

Instanciation

- Exemple:

```
$steve = new person();
$elon = new person();


$steve->set_name("Steve Jobs");
$elon->set_name("Elon Musk");

echo "Steve's full name: " . $steve->get_name();
echo "Elon's full name: " . $elon->get_name();
```

Membres statiques

- Exemple:

```php
<?php
   class Voiture{
      private static $roues=4;
      public static function message(){
         echo "Il s'agit d'une voiture.<br />";
         echo "Elle a ".self::$roues." roues.";
      }
   }

   Voiture::message();
?>
```

Constructeur

- Fonction qui permet de créer des objets et de définir leurs propriétés en une seule opération

```
void __construct($argument1,…,$argumentN)
```

- Le constructeur est une méthode :
  - dite « méthode magique » comme toutes celles qui commencent par « __ »
  - porte le même nom, quelle que soit la classe, ce qui permet des mises à jour sans avoir à modifier le nom du constructeur
  - ne retourne aucune valeur
  - appelée automatiquement lors de la création de l'objet

Constructeur

- Exemple:

```
class person {
   private $name;

   function __construct($persons_name) {
       $this->name = $persons_name;}

   function set_name($new_name) {
       $this->name = $new_name; }

   function get_name() {
       return $this->name;  }
}


$steve = new person("Steve Jobs");
echo "Steve's full name: " . $steve->get_name();
```

Constructeur

- Exemple:




```
class Motocyclette {
  //attribut
  private $couleur;

  //Constructeur
  function __construct($couleur="")    {
      echo "Constructeur<br />";
      $this->couleur = $couleur;
  }
  //etc …
}

//Déclaration d'une instance :
$MonTacot1 = new Motocylette();
//ou :
$MonTacot2 = new Motocylette("rouge");
```

Destructeur

```
void __destruct ()
```

- Le destructeur est une méthode :
  - s'utilise sans paramètre
  - ne retourne aucune valeur
  - appelée automatiquement soit après la destruction explicite de l'objet avec la fonction unset(), soit après la fin du script.
  - contient typiquement des instructions qui permettent de gérer proprement la destruction d'un objet, comme la fermeture explicite d'un fichier ou d'une connexion à une base de données

Encapsulation

- **Les modificateurs d'accessibilité**
- Il existe trois options d'accessibilité, qui s'utilisent en préfixant le nom de l'attribut ou la méthode de la classe:
- ***public***
  - Permet l'accès universel à la propriété, aussi bien dans la classe que dans tout le script, y compris pour les classes dérivées, comme vous l'avez vu jusqu'à présent.
- ***protected***
  - La propriété n'est accessible que dans la classe qui l'a créée et dans ses classes dérivées.
- ***private***
  - C'est l'option la plus stricte : l'accès à la propriété n'est possible que dans la classe et nulle part ailleurs.

L'héritage

- Permet de regrouper des parties communes dans une classe dite mère
- Toute classe dérivant de cette classe mère est appelée classe fille
- Une classe fille possède les attributs et méthodes de la classe mère ainsi que des attributs et des méthodes qui lui sont propres
- PHP ne permet pas la surcharge de fonction (ou de méthodes) et donc on ne peut attribuer le même nom à plusieurs fonctions.
- Par contre la redéfinition est possible.

L'héritage

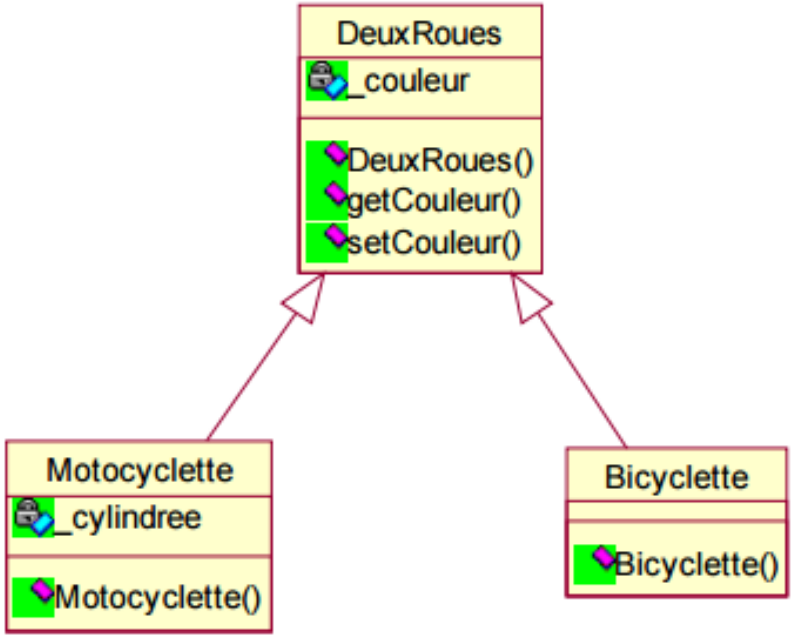
- Exemple:

```
class employee extends person {
    function __construct($employee_name) {
        $this->set_name($employee_name);
    }

}
```

- Notez que nous pouvons utiliser `set_name ()` dans `employee`, même si nous ne l'avons pas déclarer dans cette classe . C'est parce que nous avons déjà créé `set_name()` dans la classe `person`.

Exemple




```php
class DeuxRoues {
    //attribut
    private $_couleur;

    //Constructeur
    function __construct($couleur= "") {
        $this->_couleur = $couleur;
    }
    public function getCouleur() {
        return $this->_couleur;
    }
    //etc ...
}

class Bicyclette extends DeuxRoues {
    //Constructeur
    function __construct($couleur= "") {
        //Appel du constructeur parent
        parent::__construct($couleur);
    }
}
```

```php
//Déclaration d'une instance :
$MonVelo = new Bicyclette("bleue");

echo $MonVelo->getCouleur();
```

L'héritage

- **Redéfinition de méthode**
- Les méthodes hérités peuvent être réécrites dans la classe fille
- Les méthodes de la classe fille doivent avoir des prototypes compatibles avec ceux de la classe mère
  - Les classes filles doivent pouvoir se manipuler comme la classe dont elles héritent
  - Il est possible d'ajouter des paramètres supplémentaires, à condition qu'ils soient facultatifs
  - Il est aussi possible de rendre facultatifs des paramètres en leur donnant une valeur par défaut

Exemple: Redéfinition

```php
class DeuxRoues
{
        //Attribut
        protected $_couleur ;

        //Méthode
        public function getCouleur()
        {
          return $this->_couleur;
        }
}
```

```php
class Bicyclette extends DeuxRoues
{

        //redéfinition de la méthode
        public function getCouleur()
        {
           echo "Nouvelle méthode !";
           return $this->_couleur;
        }
}
```

```php
//Déclaration d'une instance :
$MonVelo = new Bicylette;

$MonVelo->getCouleur();
```

Appel de cette méthode

L'héritage

- **Résolution de portée**
- Il est possible de faire référence aux méthodes d'une classe de base.
- Pour cela, on utilise l'opérateur :: en précisant soit le nom de la classe de base soit le mot clé parent :

```
class vehicule
{
   $roues = 4;
   function affiche()
   {
    return "a ".$this->roues." roues";
   }
}
```

```
class voiture extends vehicule
{
   function affiche()
   {
    echo "cette voiture".
              parent::affiche();
   }
}
```

```
$v = new voiture();
$v->affiche(); //affiche cette voiture a 4 roues
```

Le polymorphisme

- Le polymorphisme décrit un modèle dans la programmation orientée objet dans lequel les classes ont des fonctionnalités différentes, tout en partageant une interface commune
- Les interfaces vous permettent de spécifier les méthodes qu'une classe doit implémenter.
- Les interfaces facilitent l'utilisation d'une variété de classes différentes de la même manière.
- Lorsqu'une ou plusieurs classes utilisent la même interface, on parle de "polymorphisme".
- Les interfaces sont déclarées avec le mot clé interface

Exemple:

```php
interface Animal {
  public function makeSound();
}

class Cat implements Animal {
  public function makeSound() {
    echo "Meow";  }
}
class Dog implements Animal {
  public function makeSound() {
    echo " Bark ";  }
}
class Mouse implements Animal {
  public function makeSound() {
    echo " Squeak ";  }
}
```

```php
$animal = new Cat();
$animal->makeSound();
```

Exemple:

```php
// Create a list of animals
$cat = new Cat();
$dog = new Dog();
$mouse = new Mouse();
$animals = array($cat, $dog, $mouse);

// Tell the animals to make a sound
foreach($animals as $animal) {
  $animal->makeSound();
}
```

Questions?